

ML Challenge 2026: Business Entity Resolution Solution

Team Name: Quantum · **Team Members:** Aditya Singh (leader), Achita Parmar, Akshita Singh, Anuj Gupta · **Best submission:** public leaderboard macro F0.5 **0.920** (27 Sep 2026, 11:22 PM IST)

1. Executive Summary

Our best submission is a **blocking + classifier** system that runs end to end in about 2–3 hours on an 8 GB, 2-core laptop:

1. **Normalisation:** a native-script → Latin transliteration dictionary learned from the training pairs, plus parsed name and address components.
2. **Blocking:** 13 exact keys on the *normalised* fields generate candidates, with a cap on how many S1 may share a key.
3. **Pair scoring:** a 43-feature LightGBM model scores every candidate.
4. **Decision:** a one-owner accept rule, tuned directly for macro F0.5.

It scores **0.934** out-of-fold over all 2.21 M labelled Source 1 entities (S1), and **0.920** on the public leaderboard. We also built a heavier pipeline that scores 0.974 out-of-fold (Section 6). Its test pass did not fit our compute budget, so it was not submitted.

2. Methodology

2.1 Problem Analysis

- **Scale:** 2.21 M S1, 5.03 M S2 and 5.29 M S3 training records, with 7.64 M true pairs. 5.6 % of S1 are singletons. The test set has 1.73 M S1, including France, which is unseen in training.
- **Structure:** every S2/S3 record matches **at most one** S1, and country labels always agree on true pairs. So we block within country, treating country as an open set, and enforce one owner per record.
- **Noise:** most of it is *systematic* and can be removed by normalisation:
 - Names: legal suffixes, abbreviations, | - or d/b/a-separated alternate names, spacing ("Sun Rise Traders" vs "sunrisetraders.com").
 - Scripts: Devanagari and other Indic native-script names.
 - Addresses: state abbreviations, PO boxes, landmarks.
- **Remaining noise:** typos, missing address parts, and chains whose branches share a name.

2.2 Solution Strategy

Approach type: blocking + classifier + constrained decision.

Core idea: normalise aggressively so that most true pairs become *exact* equalities on some field combination. Cheap exact joins then replace expensive fuzzy blocking, and a learned scorer separates true pairs from sibling branches.

Normalisation:

- Case and accent folding.
- Indic transliteration via Unicode character names, plus a native → Latin token dictionary learned **only from the training pairs**. No external data or APIs are used.
- Name variants: a suffix-free *core* name, a *joined* spaceless name and an *alternate* name.
- Address parsing into house number, street keys, city, state code, postal code and landmark.

Data flow: prepare (normalise) → `key_matcher.py train` (candidates, features, 2-fold model, thresholds) → `key_matcher.py test` (candidates, features, scores, decision, output/).

3. Candidate Generation (Blocking)

Blocking keys: 13 exact keys, each prefixed with the country:

Group	Keys (cap = maximum S1 that may share a key)
Name + address	name+street (3), name+house no. (3), name+city (5), name+state (5)
Name only	joined name (5), alternate name of S2/S3 = S1 core name (5) — address-less records
Address only	street+city (3), full address (3), house no.+city (3) — garbled or renamed businesses
Typo-tolerant	first 6 name chars + house / city / postal (3), last 6 name chars + house (3)

How a key links records: it links an S2/S3 record to *every* S1 that carries the same key, as long as no more than `cap` S1 share it. Keys shared by more S1 are too ambiguous and are skipped. The union over keys keeps, for each pair, which keys hit and how ambiguous they were; both are features. The joins run on dictionary-encoded Arrow columns with no Python loops, about 5 minutes for 12 M records.

Candidate pairs:

	Pairs	Per S1	Pair recall	Candidate-oracle F0.5
Train	20.26 M	9.2	0.884	0.956
Test	17.92 M	10.3	—	—

How we kept true matches:

- Each key's precision and *unique* true-pair contribution was measured on the training data. Three redundant keys added (almost) no unique true pairs and were dropped.
- True pairs found by no other key: about 460 k from the prefix/suffix keys and about 140 k from the address-only keys.
- Allowing shared keys up to the cap, and adding the typo-tolerant and address-only keys, raised pair recall from 0.62 (v1, unique keys only) to 0.88.

4. Matching Model

Features (43). No IDs, no raw strings and no country one-hot, so the model applies unchanged to France.

- **Key evidence:** 13 key-hit flags, number of keys, ambiguity, and the candidate count per S1 and per S2/S3 record.
- **Name features:** rapidfuzz token-set ratio on the core name; ratio and partial ratio on the joined name; token-sort ratio on the normalised name; name lengths.
- **Address features:** token-set ratio and ratio on the address core.
- **Agreement features:** house number, city, state and postal code — equal / conflict, plus a postal-code-missing flag.
- **Rival features:** a pair's name and address similarity minus the best among the same S1's, and the same S2/S3 record's, other candidates.
- **Source flags:** S3 vs S2, empty address on either side, web-style name, native-script name.

Model type: LightGBM binary classifier (MIT licence): 300 trees, 127 leaves, learning rate 0.1, 63 bins. Training uses 2-fold GroupKFold by S1 over all labelled S1. Each fold model scores the half it did not see; these out-of-fold scores are used for evaluation and threshold tuning. Test pairs are scored with the **average of the two fold models**, which together were trained on all labelled data.

Threshold selection (decision layer):

1. Each S2/S3 record keeps only its best-scoring S1 (one owner).
2. An S1 is "opened" if its best owned candidate has $p \geq t_{\text{open}}$. It then accepts every owned candidate with $p \geq t_{\text{add}}$.
3. t_{open} and t_{add} were grid-searched on the out-of-fold scores to maximise macro F0.5, singletons included, separately per country: US 0.60 / 0.65 and India 0.60 / 0.70. Unseen countries (France) use the global values, 0.60 / 0.65.

5. Results & Error Analysis

Version (macro F0.5)	US	India	All (out-of-fold)	Leaderboard
v1: unique exact keys, no model	0.838	0.678	0.774	0.762
v2: key candidates + LightGBM (best submission)	0.957	0.899	0.934	0.920
v2 + rival-score second stage	0.952	0.896	0.929	not submitted
Full pipeline (Section 6; 30 % S1 sample)	0.976	0.970	0.974	test pass not finished

v2 has pair precision **0.989** and recall 0.853, and leaves 94.4 % of singletons correctly empty. The 0.014 gap to the leaderboard most likely comes from France, which is absent from training.

- **False positives (wrong merges), 75 k out-of-fold:** in 62 % the two records share the same core name, i.e. a *sibling branch of a chain*. 31 % have conflicting house numbers.
- **False negatives (missed matches), 1.13 M:**
 - **79 % never became candidates.** In 84 % of these the joined names differ (typos, abbreviations, reordering), and in 69 % the city differs or is missing. Exact keys cannot reach them.
 - The remaining 21 % were candidates but rejected. 81 % of those have identical core names, and 53 % also show a house-number conflict. The model could not tell which of several same-name branches the record belongs to.

6. Conclusion and Further Work

Aggressive normalisation turns most of the noise into exact equalities. That let a cheap key-join blocker and a small LightGBM reach 0.92 on the leaderboard within a laptop's budget. Our main limit is blocking recall (0.884), not scoring.

We also built the fuller pipeline, sharing the same normalisation, folds and decision layer:

- **Blocking:** ten hashed TF-IDF sparse top-k blocking passes covering name, name × address, address pairs, phonetic skeletons, name prefixes, exact address, and a reverse pass for address-less records.
- **Pruning:** a monotone-constrained LightGBM pre-ranker with MinHash record signatures, then a cheap-feature ranker. Each pruning step was accepted only if it lost ≤ 0.0005 of oracle F0.5.
- **Matcher:** 101 features.
- **Results:** it raises the candidate ceiling from 0.956 to **0.9956** and reaches **0.974** out-of-fold. Its test pass takes about 6 hours on our hardware.

Lesson: budget test-time inference as early as training.

Appendix

A. Code Artefacts

The code is in `code/business_entity_resolution/`: all source in `src/`, with a `README.md` and a pinned `requirements.txt` (Python 3.13; numpy, pandas, pyarrow, scipy, scikit-learn, LightGBM 4.6, rapidfuzz). Place the competition data in `dataset/train` and `dataset/test`, then run from `src/`:

1. `python run_pipeline.py prepare` (~1 h): TSV → parquet; learn the transliteration dictionary; write normalised Arrow tables.
2. `python key_matcher.py train` (~45 min): training candidates and features; 2-fold models; out-of-fold F0.5 and thresholds.
3. `python key_matcher.py test` (~30 min): writes `output/matching_results.tsv` and `output/candidate_pairs.tsv`.

`candidate_pairs.tsv` is the exact candidate set the model scores, and every matched ID is one of its candidates.

Other source files:

- `run_pipeline.py`: the full pipeline from Section 6 (`ER_MODE=FINAL python run_pipeline.py auto`).
- `fast_fallback.py`: v1.
- `key_stage2.py`: the unused second stage.